

Basic (a normal, exact test) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper pytest, including edge cases a hand-picked example misses). Read Basic first even if you already know pytest — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

`chunk_by_paragraph` is a plain Python function — no LLM involved. That means you can test it the normal way: give it a known input, and check the output is exactly what you expect.

A pytest test is just a function whose name starts with `test_`. Inside it, you use plain `assert` statements. If the assert is true, the test passes silently. If it's false, pytest shows you exactly what didn't match.

Think about what you already know for sure: if you feed the function a piece of text with 3 paragraphs separated by blank lines, you already know the answer should be 3 chunks — before you even run the code.

Intermediate Version

This exercise is really about the difference between testing *deterministic* code (same input, same output, every time) and the LLM-touching tests later in this document. `chunk_by_paragraph` has no model call inside it, so an exact `assert` is completely right here — don't reach for anything fuzzier.

A count check alone is weak: `len(chunks) == 3` still passes if the chunker scrambles the order or cuts off the last sentence. So also check the *content* of the first and last chunk. And one neat example only proves the function works on the input you thought of — real documents have messy text: one paragraph with no blank lines, extra blank lines in a row, or an empty file.

The exact pieces:

- Copy Doc08's `chunking.py` (the one with `chunk_by_paragraph`) into `practice/`, unchanged.
- Import the function you're testing: `from chunking import chunk_by_paragraph`.
- Run it with `pytest chunking_unit_test_practice.py -v` from inside `practice/` — naming the file on the command line makes pytest collect every `test_` function in it.
- A second test function for the single-paragraph case — one clearly named test per case.
- `@pytest.mark.parametrize("text,expected_count", [...])` — runs the same test body once per row of a table, and reports each row as its own pass/fail. Good for many small cases (empty string, extra blank lines) without writing a new function for each.

Difference between Basic and Intermediate: Basic describes the plain idea for one tidy, hand-picked example. Intermediate checks content as well as count, adds a second test for a messy input, and uses `parametrize` to run a whole table of edge cases through one test body.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
import chunk_by_paragraph from chunking

make sample_text = a string with 3 paragraphs, separated by blank lines

def test_chunk_by_paragraph():
    chunks = chunk_by_paragraph(sample_text)
    assert there are exactly 3 chunks
```

Here's almost the whole thing — just try running it and reading it line by line:

```
# practice/chunking_unit_test_practice.py
from chunking import chunk_by_paragraph

sample_text = "Paragraph one.\n\nParagraph two.\n\nParagraph three."

def test_chunk_by_paragraph():
    chunks = chunk_by_paragraph(sample_text)
    assert len(chunks) == 3
```

Expected output: run with `pytest chunking_unit_test_practice.py -v` — you should see `test_chunk_by_paragraph PASSED`.

Intermediate Version

```
same import and sample_text

def test_chunk_by_paragraph():
    check the count, the first chunk's text, and the last chunk's text

def test_chunk_by_paragraph_single_paragraph():
    one paragraph with no blank lines -> exactly 1 chunk, same text

parametrize a table of (text, expected_count):
    3 paragraphs -> 3, one paragraph -> 1, "" -> 0, extra blank lines -> 2
```

Here's most of it — fill in the table yourself:

```
# practice/chunking_unit_test_practice.py
import pytest
from chunking import chunk_by_paragraph

sample_text = "Paragraph one.\n\nParagraph two.\n\nParagraph three."

def test_chunk_by_paragraph():
    chunks = chunk_by_paragraph(sample_text)
    assert len(chunks) == 3
    assert chunks[0] == "Paragraph one."
    assert chunks[-1] == "Paragraph three."

@pytest.mark.parametrize(
    "text,expected_count",
    [
        # your turn: add rows like ("", 0) and ("A.\n\n\n\nB.", 2)
    ],
)
def test_chunk_by_paragraph_count(text, expected_count):
    chunks = chunk_by_paragraph(text)
    assert len(chunks) == expected_count
```

Before you add a row, decide what the *right* answer is — then check whether Doc08's chunker agrees.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)